

Based on your profile as a **Full Stack .NET Developer** with 2 years of experience and the technical standards of mid-tier companies in the Noida region, here is a comprehensive 100-step roadmap. This guide covers everything from the initial "intro" to advanced technical questions with code and final salary negotiations.

## Phase 1: The First Impression (Questions 1–10)

*Mid-tier companies in Noida value stability and a "can-do" attitude.*

1. **Tell me about yourself?** Focus on your transition from B.Tech at MG Institute to your work at Vijatshi Software. Mention your core stack: .NET Core, SQL Server, and AngularJS.
2. **Why are you leaving your current role?** Focus on seeking new challenges in architecture or larger-scale database optimization.
3. **What was your role in the "Loyalty Payback Program"?** Explain how you used .NET and AngularJS for real-time tracking.
4. **How do you handle deadlines?** Mention your experience delivering modules like HR portals or ERP integrations.
5. **What is your experience with legacy systems?** Talk about modernizing ASP.NET MVC systems to the latest .NET frameworks.
6. **Are you comfortable working in Noida/Delhi?** (Verify your current base in Noida).
7. **What is your notice period?** (Be honest; 30–60 days is standard for mid-tier firms).
8. **How do you stay updated with tech?** Mention following .NET Core 6/8 updates.
9. **Describe a difficult bug you solved.** Mention your experience with SQL performance tuning.
10. **Do you prefer frontend or backend?** As a Full Stack developer, emphasize that you are comfortable with both but specialize in API/DB logic.

---

## Phase 2: C# & .NET Core Fundamentals (Questions 11–35)

*Tier-2 companies often test your understanding of memory and basics.*

11. **Value vs. Reference Types?** Value types (int, struct) live on the Stack; Reference types (class, string) live on the Heap.
12. **Boxing and Unboxing?** Boxing is converting a value type to an object; unboxing is the reverse.

13. **Why is StringBuilder better than String for loops?** Strings are immutable; modifying them creates new objects. StringBuilder is mutable and uses an internal buffer.

C#

```
var sb = new StringBuilder();  
for(int i=0; i<100; i++) sb.Append(i); // Efficient [cite: 544]
```

14. **Managed vs. Unmanaged Code?** Managed code runs under the CLR; Unmanaged code (like C++) requires manual memory management.
15. **What are the GC Generations?** Gen 0 (new), Gen 1 (buffer), Gen 2 (long-lived).
16. **Ref vs. Out?** ref must be initialized before passing; out must be assigned inside the method.
17. **IEnumerable vs. List?** IEnumerable is read-only and uses lazy evaluation. List is for manipulation and loads everything into memory.
18. **Interface vs. Abstract Class?** Interfaces are contracts (no body in old versions). Abstract classes can have implementation.
19. **What is a "Record" in C#?** Immutable by default, using value-based equality.
20. **Const vs. Readonly?** Const is compile-time; Readonly can be assigned at runtime in a constructor.
21. **What is the using statement?** Ensures IDisposable objects are cleaned up immediately.
22. **What is Reflection?** Inspecting metadata at runtime.
23. **Why is Reflection slow?** CLR performs security checks and searches by string names.
24. **Difference between Task and Thread?** A Thread is a heavy OS-level object; a Task is a higher-level abstraction managed by the ThreadPool.
25. **What is the yield return keyword?** Creates custom iterations without storing the whole list in memory.
26. **Explain Extension Methods.** Adding methods to existing classes without inheritance.
27. **What is an Indexer?** Allows an object to be treated like an array.

- 28. **Access Modifiers?** Public, Private, Protected, Internal.
- 29. **Difference between var and dynamic?** var is static/compile-time; dynamic is run-time.
- 30. **What is an Interface Default Implementation?** Since C# 8, interfaces can have default logic.
- 31. **What are Delegates?** Type-safe function pointers.
- 32. **Action vs. Func?** Action returns void; Func returns a value.
- 33. **What is a Constructor?** Initializes an object.
- 34. **Can a Constructor be private?** Yes, used in Singleton patterns.
- 35. **What is Object-Oriented Programming (OOP)?** Encapsulation, Inheritance, Polymorphism, and Abstraction.

---

## Phase 3: SQL & Database Optimization (Questions 36–55)

Your profile emphasizes SQL performance tuning.

- 36. **Primary Key vs. Unique Key?** Primary (1 per table, no NULLs). Unique (Multiple allowed, 1 NULL allowed).
- 37. **DELETE vs. TRUNCATE?** DELETE is logged row-by-row and allows WHERE. TRUNCATE is faster and removes everything.
- 38. **What are ACID properties?** Atomicity, Consistency, Isolation, Durability.
- 39. **What is a CTE?** A temporary result set, often used for readability or recursion.
- 40. **WHERE vs. HAVING?** WHERE filters rows before grouping; HAVING filters groups.
- 41. **Stored Procedure (SP) benefits?** Security (prevents SQL injection), performance (pre-compiled), and reduced network traffic.
- 42. **What is a Trigger?** Code that runs automatically on INSERT/UPDATE/DELETE.

43. **What is an Execution Plan?** A graphical representation of how SQL executes a query.
44. **Index Scan vs. Index Seek?** Scan (Slow, looks through all) vs. Seek (Fast, uses index directly).
45. **What is an Indexed View?** A view with a unique clustered index that physically stores results on disk.
46. **SQL Isolation Levels?** Read Uncommitted, Read Committed (Default), Repeatable Read, Serializable.
47. **EXISTS vs. IN?** EXISTS is usually faster for subqueries because it short-circuits on the first match.
48. **How to find the 2nd Highest Salary?**  
SQL  
`SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary) FROM Employees);` [cite: 20, 21]
49. **What is a Scalar Function vs. Table-Valued?** Scalar returns a single value; Table-valued returns a result set.
50. **SQL Normalization (1NF, 2NF, 3NF)?** Rules to remove redundancy and transitive dependency.
51. **What is a SQL Deadlock?** When two transactions lock each other; solve using DEADLOCK\_PRIORITY.
52. **CROSS APPLY?** Used to join a table with a function.
53. **NoSQL vs. SQL?** Use NoSQL (MongoDB) for unstructured data and SQL for relational data.
54. **How to prevent SQL Injection in C#?** Always use Parameterized Queries or Stored Procedures.
55. **What is the Large Object Heap (LOH)?** Stores objects > 85KB; not compacted by the GC.

---

## Phase 4: ASP.NET Core & Web API (Questions 56–75)

Focus on RESTful APIs and middleware.

- 56. **What is Middleware?** Components in the request pipeline.
- 57. **\_next(context) delegate?** Passes the request to the next middleware; if not called, the request short-circuits.
- 58. **JWT Authentication?** Stateless token-based security consisting of Header, Payload, and Signature.
- 59. **AddScoped vs. AddTransient vs. AddSingleton?**
  - **Transient:** New every time.
  - **Scoped:** New per HTTP request.
  - **Singleton:** One for the entire application life.
- 60. **HttpPut vs. HttpPatch?** PUT replaces the whole object; PATCH is for partial updates.
- 61. **What is CORS?** Security feature preventing cross-domain requests.
- 62. **What is Idempotency?** Multiple identical requests returning the same result (GET, PUT, DELETE).
- 63. **Content Negotiation?** Client uses "Accept" header to request specific formats like JSON or XML.
- 64. **Swagger (OpenAPI)?** Used for documenting your API.
- 65. **Thin vs. Fat Controller?** Move business logic to a Service Layer (Thin) instead of keeping it in the Controller (Fat).
- 66. **What is the \$N+1\$ query problem?** Common EF issue where fetching children in a loop causes multiple SQL queries.
- 67. **How to solve \$N+1\$?** Use .Include() for Eager Loading.
- 68. **When to use AsNoTracking()?** For read-only reports to improve performance and save memory.
- 69. **Code First vs. Database First?** Code First (classes build the DB) vs. Database First (DB

generates models).

- 70. **What are EF Migrations?** Version control for your database schema.
- 71. **How to execute Raw SQL in EF Core?** Use `db.Database.SqlQuery` for complex SPs.
- 72. **What is CQRS?** Separating Read and Write operations.
- 73. **Rate Limiting?** Preventing API abuse by limiting calls.
- 74. **Environment Variables?** Keeping secrets (like keys) out of `appsettings.json`.
- 75. **What is SignalR?** Real-time bi-directional communication using WebSockets.

---

## Phase 5: Design Patterns & Architecture (Questions 76–85)

Essential for proving seniority at the 2-year mark.

- 76. **Singleton Pattern?** Ensuring only one instance of a class exists (use a private constructor).
- 77. **Repository Pattern?** Centralizing data access logic.
- 78. **Unit of Work?** Managing multiple repository changes in a single transaction.
- 79. **Factory Pattern?** Hiding object creation logic from the client.
- 80. **Observer Pattern?** Notifying subscribers automatically when an object's state changes.
- 81. **Circuit Breaker Pattern?** Stops a microservice from calling a failing service to prevent cascading failure.
- 82. **Saga Pattern?** Manages distributed transactions across microservices using compensating transactions.
- 83. **Blue-Green Deployment?** Using two identical environments to switch traffic during updates.
- 84. **What are SOLID principles?** Five design principles (S-O-L-I-D) for clean, maintainable

code.

85. **Dependency Inversion?** Depending on interfaces rather than concrete classes.

---

## Phase 6: Frontend & Tools (Questions 86–95)

Your profile includes AngularJS and Blazor.

86. **AngularJS vs. .NET Core integration?** Explain how you built dynamic dashboards to fetch real-time data.

87. **What is Role-Based Access Control (RBAC)?** Implementing security based on user roles (e.g., in your HR project).

88. **What is Blazor?** C# frontend framework for building interactive web UIs.

89. **WebAssembly (WASM) vs. Server-side Blazor?** Client-side execution vs. SignalR-based server execution.

90. **AutoMapper?** Automation tool to convert domain objects into DTOs to reduce boilerplate.

91. **Unit Testing AAA Pattern?** Arrange, Act, Assert.

92. **Mocking vs. Stubbing?** Stubs provide hardcoded data. Mocks also verify if methods were called.

93. **What is TDD?** Test Driven Development: Write the test first, then the code.

94. **Git Workflow?** Explain your use of Git/GitLab for version control at Vijatshi.

95. **Status Codes (200, 401, 404, 500)?** OK, Unauthorized, Not Found, Server Error.

---

## Phase 7: Salary & Negotiation (Questions 96–100)

*Noida mid-tier companies often have budget caps; know your value.*

96. **What are your salary expectations?** Research the market for "2 years experience .NET Developer in Noida." Typically, this ranges from **₹6 LPA to ₹10 LPA** depending on skills.

97. **Can you justify your hike?** Mention your "Full Stack" capability , database optimization skills , and experience with modern frameworks like .NET 8.
98. **Are you open to a technical round on-site?** Most Noida firms prefer one on-site round.
99. **When can you join?** If you are an immediate joiner, you have more leverage in salary negotiation.
100. **The "Human" Factor:** Remember the best developer trait: "Writing code for humans to read, not just for machines to run".